

Keywords: tool calling • LLM agents • programmatic invocation • benchmark evaluation

The Bitter Lesson of Tool Calling

Programmatic Python-Stub Invocation Outperforms JSON Tool Calling in 11 of 14 Models Across a Generation-Spanning Benchmark

By Ishan Patel, Sahil Sen, Elias Lumer, Vamse Kumar Subbiah
arXiv:2608.06370 • Preprint, August 2026

The dominant paradigm for giving language models access to external tools—native JSON tool calling—was adopted by convention, not by systematic study. A new paper comparing Programmatic Tool Calling (PTC) against JSON tool calling across 14 models on BFCL v4 finds that PTC matches or beats JSON in 11 of 14 models, with the GPT-5.6 family gaining 10.6% and PTC winning 13 of 14 models under parallel fan-out tasks.

AT A GLANCE

Problem: The JSON tool-calling format is the default for LLM agents, but has never been systematically compared to programmatic alternatives at scale.

Why it matters: Tool calling underlies virtually every production LLM agent; a 10% accuracy gap from a format choice is a free performance gain.

Method: Evaluate PTC (typed Python stubs + code execution) vs. JSON across 14 models on BFCL v4 under three tracks: standard, parallel fan-out, context rot.

Result: PTC wins in 11/14 standard, 13/14 parallel fan-out; JSON drops 2.3% under context rot while PTC holds steady.

Evidence: GPT-5.6 family gains +10.6% with PTC; results replicated across model families and generations.

The Big Picture

Every production LLM agent system must make a choice the field has rarely examined: how should tools be presented to the language model? The two dominant options are **native JSON tool calling**, where the model emits a structured JSON object naming the tool and providing argument values, and **Programmatic Tool Calling (PTC)**, where tools are defined as typed Python function stubs and the model writes executable Python code to invoke them.

JSON tool calling became the standard through historical momentum—it was the format exposed by early commer-

cial APIs, and the ecosystem solidified around it. This paper asks whether that default is actually optimal, and finds systematically that it is not. PTC exploits models’ extensive code training, allows the model to reason about function signatures before committing to a call, and composes naturally with Python’s type system for argument validation.

Why Existing Approaches Fall Short

JSON tool calling has two structural weaknesses that PTC avoids. First, the JSON schema for a tool is typically serialized as a string in the prompt, forcing the model to parse an ad hoc format rather than the typed function signatures it encountered during code pretraining. Second, under parallel fan-out—where multiple tools must be invoked in a single turn—JSON requires emitting a list of JSON objects, a format the model sees rarely, whereas PTC naturally supports multi-function Python calls with loop and control-flow structures.

Under **context rot**—the realistic condition where prior conversation turns accumulate in the context window, cluttering it with tool definitions and results from earlier steps—JSON performance degrades 2.3% on average while PTC holds steady. The hypothesis is that Python code provides stronger structural anchoring than JSON blobs, making it more robust to noisy context.

Core Mechanism

In PTC, each tool is presented as a typed Python stub:

```
def search_web(query: str,
               max_results: int = 5
               ) -> list[dict]:
    """Search the web for query."""
    ...
```

The model is instructed to write Python code invoking these stubs, with actual execution handled by the agent harness. Results are returned as Python objects and made available to the next step. This mirrors the model’s training distribution for code: function calls, not JSON blobs.

Technical Formulation

Let $\mathcal{T} = \{t_1, \dots, t_K\}$ be the set of available tools and q the user query. Under JSON calling, the model produces:

$$a_{\text{JSON}} = \{ \text{"name"} : t_k, \text{"args"} : \{ \dots \} \}$$

Under PTC, the model produces executable Python:

$$a_{\text{PTC}} = t_k(\text{arg}_1 = v_1, \dots, \text{arg}_m = v_m)$$

Functional correctness is measured on BFCL v4 by checking that a matches the reference invocation: $\mathbb{K}[a = a^*]$.

The aggregate metric is accuracy averaged over all items in each evaluation track.

The context-rot track injects n prior turns of tool-use history into the context before the test query:

$$\Delta_{\text{rot}} = \text{Acc}(n = 0) - \text{Acc}(n = N_{\text{rot}})$$

with $\Delta_{\text{rot}}^{\text{PTC}} \approx 0$ and $\Delta_{\text{rot}}^{\text{JSON}} \approx -2.3\%$ on average.

Learning or Inference Procedure

No training. Both PTC and JSON are evaluated at inference time by changing only the prompt format and tool-presentation schema. No fine-tuning is performed. The study is therefore a pure format comparison, isolating the effect of tool presentation from any model-level differences.

BFCL v4 provides three tracks:

- **Standard:** single-turn, well-scoped tool calls
- **Parallel fan-out:** multiple tools invoked in one turn
- **Context rot:** accumulated prior turns in context

All 14 models are evaluated identically under both formats.

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that PTC’s advantage is consistent across 14 models spanning multiple families and training generations, making it unlikely to be an artifact of any single model’s quirks. The parallel fan-out win rate of 13/14 is particularly robust. The paper does not claim PTC universally dominates, but establishes a strong empirical prior for practitioners choosing a tool-calling format.

Experimental Findings

- **Standard track:** PTC matches or beats JSON in 11 of 14 models.
- **Parallel fan-out:** PTC wins in 13 of 14 models—the largest margin.
- **Context rot:** JSON accuracy drops 2.3% on average; PTC accuracy is stable.
- **GPT-5.6 family:** +10.6% with PTC over JSON baseline—the largest single-model gain observed.
- **Failure cases:** PTC loses in 3/14 models on the standard track; all three are models with weaker code-generation training.

Ablations and Interpretation

Why does PTC help on parallel fan-out? PTC naturally expresses multiple calls as sequential Python statements or a list, both of which appear extensively in code pretraining. JSON parallel calls require a list of

JSON objects, a format the model sees infrequently.

Why is PTC robust to context rot? Python code has clear syntactic structure (indentation, keywords, type annotations) that provides stronger anchoring than JSON when the context window is cluttered.

Why does PTC fail in 3 models? The three models where JSON wins have weaker code-generation capability, suggesting the PTC advantage is mediated by code training. For code-weak models, the structured JSON prompt may be clearer than free Python code.

Practical takeaway: For any model with strong code-generation training—which describes the majority of frontier LLMs in 2026—switching from JSON to PTC is a zero-cost accuracy gain.

Reference

Ishan Patel, Sahil Sen, Elias Lumer, Vamse Kumar Subbiah. **The Bitter Lesson of Tool Calling.** arXiv:2608.06370, August 2026. <https://arxiv.org/abs/2608.06370>

Keywords: context reliability • sycophancy • direct preference optimization • LLM alignment

Learning When to Trust via Selective Context Preference Optimization

MIST Benchmark and SCOPE Training Teach Language Models to Resist Misleading Context Without Ignoring Helpful Signals

By Xian Sun, Wei Chow, Yingshuo Wang, Junhao Liu, Wei Gao, Qing Wu, Lingdong Kong

arXiv:2608.06377 • Preprint, August 2026

A new paper introduces MIST, a benchmark that renders each reasoning item under four matched context conditions, and SCOPE, a DPO-based training method that teaches models to distinguish misleading from helpful context. SCOPE significantly reduces the fraction of correct answers flipped wrong by misleading signals while preserving the model’s benefit from genuinely useful context.

AT A GLANCE

Problem: LLMs are sycophantic toward misleading context yet training to resist context can make them ignore helpful signals.

Why it matters: RAG, web search, and chat history all deliver context that may be wrong; models that cannot discriminate are unsafe to deploy.

Method: MIST benchmark (4 matched conditions per item) + SC2W metric + SCOPE (DPO over balanced preference pairs across all four conditions).

Result: SCOPE significantly reduces SC2W while preserving correct-context utilization; balanced training is crucial.

Evidence: Ablation confirms that optimizing only on misleading pairs degrades helpful-context performance.

The Big Picture

Retrieval-augmented generation, web-search-augmented chat, and memory-augmented agents all place LLMs in an environment where external context is omnipresent. When that context is accurate, it should help the model answer correctly. When it is misleading—outdated retrieval, user misconceptions, adversarial injections—the model should recognize the discrepancy and trust its own knowledge over the flawed signal.

Current LLMs fail in both directions: they are sycophantic toward user-provided context (a misleading signal flips a correct answer wrong), yet attempts to harden models

against misleading context often produce models that ignore helpful context universally. This paper frames the problem as a **discrimination task**: the model must learn to distinguish the quality of context, not merely resist all of it.

Why Existing Approaches Fall Short

Prior robustness work has largely attacked misleading context as a single phenomenon, training on adversarial examples where context is always wrong. This produces models with one failure mode traded for another: high resistance to misleading context, but equally high resistance to correct context, because the model has learned “context → ignore,” not “misleading context → ignore.”

Evaluating this nuance requires a benchmark that explicitly presents both misleading and correct context for the same item. Without matched conditions, it is impossible to separate context-resistance from context-utilization, and prior evaluations have conflated the two.

Core Mechanism

MIST Benchmark. Each item in MIST appears under four matched conditions:

1. **Clean:** question alone, no external context
2. **Misleading:** context that nudges toward the wrong answer
3. **Correct context:** context that supports the right answer
4. **Irrelevant:** context unrelated to the question

The **SC2W metric** (Susceptibility to Context-induced Wrong answers) counts how often condition 2 flips a clean-correct answer to wrong:

$$\text{SC2W} = \frac{\#\{\text{clean-correct} \wedge \text{misleading-wrong}\}}{\#\{\text{clean-correct}\}}$$

SCOPE Training. SCOPE (Selective Context Preference Optimization) applies DPO over preference pairs mined from MIST failures. The preferred completion is the model’s clean-correct answer; the dispreferred completion is the misleading-flipped wrong answer. Crucially, pairs are sampled **equally across all four conditions**, not only misleading ones.

Technical Formulation

Let x be a question and c a context drawn from condition set $\mathcal{C} = \{\emptyset, \text{mis}, \text{cor}, \text{irr}\}$. The DPO objective over preference pairs (y_w, y_l) is:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,c,y_w,y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x, c)}{\pi_{\text{ref}}(y_w | x, c)} - \beta \log \frac{\pi_{\theta}(y_l | x, c)}{\pi_{\text{ref}}(y_l | x, c)} \right) \right]$$

where $\mathcal{D}$ is the balanced dataset spanning all four conditions with equal weight. The SC2W constraint requires that:

$$\mathbb{P}[y^* \mid x, c_{\text{mis}}] > \mathbb{P}[y^* \mid x, c_{\text{mis}}] \Big|_{\pi_{\text{ref}}}$$

while preserving:

$$\mathbb{P}[y^* \mid x, c_{\text{cor}}] \approx \mathbb{P}[y^* \mid x, c_{\text{cor}}] \Big|_{\pi_{\text{ref}}}$$

Learning or Inference Procedure

Pair mining: Identify items where the base model is correct in the clean condition but incorrect in the misleading condition. Each such item yields one DPO preference pair.

Balanced sampling: Sample equal numbers of preference pairs from each of the four conditions. This ensures the model cannot learn a context-agnostic “ignore all signals” policy.

DPO fine-tuning: Standard DPO applied to the preference dataset; no modifications to the DPO algorithm are required.

Inference: No change. The trained model processes queries with external context in the standard RAG pipeline.

What the Guarantee Says

No formal guarantee. The balanced training design provides an intuitive argument: because pairs from the correct-context condition reward trusting helpful context while pairs from the misleading condition reward resisting bad context, the model must learn the discrimination rather than a blanket policy. The empirical ablation confirms this: training on misleading-only pairs degrades correct-context utilization, while balanced training does not.

Experimental Findings

- **SC2W reduction:** SCOPE significantly reduces the fraction of correct answers flipped wrong by misleading context.
- **Correct-context utilization:** Preserved after SCOPE training; the model continues to benefit from genuine context.
- **Irrelevant-context:** Model correctly learns to ignore irrelevant signals.
- **Clean accuracy:** Maintained; SCOPE does not degrade the model’s baseline reasoning.

Ablations and Interpretation

Misleading-only DPO vs. SCOPE. Training DPO only on misleading-context pairs achieves strong SC2W

reduction but degrades correct-context utilization by a significant margin, confirming the “context-ignore” failure mode. SCOPE’s balanced sampling resolves this.

Effect of condition balance. Increasing the proportion of misleading pairs toward 100% monotonically trades SC2W improvement for correct-context degradation. The balanced 50/50/50/50 split across conditions achieves the best combined performance.

MIST item difficulty. High-SC2W items tend to involve superficially plausible misleading contexts (wrong dates, plausible-but-incorrect numbers). These are the cases where the model most needs the discrimination capability SCOPE provides.

Reference

Xian Sun, Wei Chow, Yingshuo Wang, Junhao Liu, Wei Gao, Qing Wu, Lingdong Kong. **Learning When to Trust via Selective Context Preference Optimization.** arXiv:2608.06377, August 2026. <https://arxiv.org/abs/2608.06377>

Keywords: legged robotics • reinforcement learning • spatiotemporal control • dynamic interception

Spatiotemporal Agility: Time-Constrained Rein- forcement Learning for Vision-Guided Dynamic Quadrupedal Interception

Zhejiang University Team Replaces Velocity-Tracking Policies with Direct Temporal-Spatial Conditioning to Enable Quadruped Ball-Catching Under Strict Deadlines

By Yidong Zhu, Zibo Dai, Tongning Zhang, Leixin Chang, Hua Chen (Zhejiang University; LimX Dynamics)

arXiv:2608.06907 • Preprint, August 2026

A quadruped robot catching a thrown ball must predict where and when the ball lands from visual observations, then reach that position before the deadline. A new paper from Zhejiang University and LimX Dynamics argues that the standard velocity-tracking locomotion paradigm is fundamentally mismatched to this class of tasks, and proposes a unified framework that conditions an RL policy directly on spatial targets and temporal deadlines rather than velocity commands.

AT A GLANCE

Problem: Velocity-tracking quadruped policies cannot meet strict temporal deadlines for reaching precise positions in dynamic interaction tasks.

Why it matters: Dynamic interaction (catching, intercepting, dodging) is the next frontier for legged robot agility; no existing policy architecture handles it.

Method: Vision module predicts landing point and time; RL policy is conditioned on (position, deadline) rather than velocity commands.

Result: Ball-catching benchmark significantly outperforms velocity-tracking decomposition baselines.

Evidence: Evaluated on a standardized ball-catching task with randomized trajectory and timing.

The Big Picture

Legged robots have progressed from laboratory curiosities to practical platforms over the past decade, largely driven by RL-based locomotion policies that excel at navigating diverse terrain. But the benchmark tasks that drove this progress—walking on uneven ground, stair climbing, recovering from pushes—share a common structure: they

reward stable navigation with no hard temporal deadline.

Dynamic interaction tasks are qualitatively different. A robot catching a ball, intercepting a rolling object, or dodging a thrown projectile must solve a joint **spatiotemporal problem**: arrive at a specific location by a specific time. Velocity-tracking policies, which accept a desired velocity vector and try to track it, have no native representation of temporal deadlines. Even if they could in principle navigate to any position, they cannot guarantee arrival by a deadline, making them unsuitable for dynamic interception.

Why Existing Approaches Fall Short

The standard decomposition for legged robot tasks is:

- **High-level planner:** computes a path and velocity profile
- **Low-level controller:** tracks the desired velocity command

This decomposition fails for time-constrained interception for two reasons. First, velocity-tracking introduces an abstraction layer that discards the temporal deadline: the planner must guess a velocity profile that achieves arrival by the deadline, but the tracking error of the low-level controller means the actual arrival time is unpredictable.

Second, the two-level decomposition requires the planner to predict the controller’s behavior, introducing compounding errors. In practice, systems that work well for slow, navigational tasks become brittle when deadlines are tight and the target is moving.

Core Mechanism

Vision Module. A neural vision module processes monocular camera images to predict:

- $\hat{\mathbf{p}} \in \mathbb{R}^3$: predicted landing position (world frame)
- $\hat{\tau} \in \mathbb{R}_{>0}$: predicted time-to-landing (seconds)

The module is trained on ball-trajectory data with supervised regression.

Time-Conditioned RL Policy. Rather than consuming velocity commands, the locomotion policy is conditioned on the pair $(\hat{\mathbf{p}}, \hat{\tau})$. This directly expresses the spatiotemporal constraint: reach position $\hat{\mathbf{p}}$ within $\hat{\tau}$ seconds. The policy must learn internally to compute appropriate gaits, step frequencies, and postures to meet both the spatial and temporal objectives simultaneously.

Technical Formulation

Let the robot state at time t be s_t and the action (joint torques or velocity targets) be a_t . The policy $\pi_\theta(a_t \mid s_t, \hat{\mathbf{p}}, \hat{\tau} - t)$ is conditioned on the remaining time budget $\hat{\tau} - t$, which decreases with each timestep, providing an

implicit urgency signal.

The reward is:

$$r_t = \lambda_{\text{pos}} r_{\text{pos}}(s_T, \hat{\mathbf{p}}) + \lambda_{\text{time}} r_{\text{time}}(T, \hat{\tau}) + \lambda_{\text{stab}} r_{\text{stab}}(s_t)$$

where T is the actual interception time, r_{pos} rewards proximity to the landing point, r_{time} penalizes arriving after the deadline, and r_{stab} encourages stable locomotion.

The vision module and locomotion policy are trained jointly, with the RL reward backpropagating a signal to align vision predictions with achievable interception targets.

Learning or Inference Procedure

Simulation training: The robot and ball physics are simulated with randomized launch parameters (speed, angle, direction) to cover diverse trajectory families. Domain randomization is applied to robot dynamics to improve sim-to-real transfer.

Curriculum: Training begins with slow, predictable ball trajectories and gradually increases speed and variability as the policy improves.

Inference: The vision module processes each incoming frame, updating $\hat{\mathbf{p}}$ and $\hat{\tau}$ in real time. The locomotion policy uses the latest prediction to compute joint targets at 50 Hz.

What the Guarantee Says

No formal guarantee is provided. The policy’s behavior near the deadline depends on learned heuristics rather than provable time-optimal control. However, the experimental results demonstrate that the time-conditioned formulation produces significantly more reliable deadline adherence than velocity-tracking baselines, suggesting the implicit urgency signal is effective in practice.

Experimental Findings

- **Ball-catching success rate:** The time-conditioned RL policy significantly outperforms velocity-tracking decomposition baselines on the proposed benchmark.
- **Deadline adherence:** The policy learns to vary gait speed continuously based on remaining time, rather than running at maximum speed and stopping.
- **Trajectory diversity:** Performance degrades gracefully with increasing ball speed and trajectory variability.
- **Sim-to-real:** The framework transfers to physical hardware with domain randomization, demonstrating real-world viability.

Ablations and Interpretation

Time conditioning vs. position-only conditioning.

Removing the time dimension and conditioning only on target position degrades success rate substantially, confirming that the temporal deadline is a necessary conditioning input rather than information the policy can infer from position alone.

Joint vs. separate training. Training the vision module and locomotion policy separately (first vision, then RL with fixed vision) produces lower success rates than joint training, suggesting that the locomotion policy’s behavior affects what landing predictions are achievable.

Curriculum vs. full distribution. Training without curriculum (full ball-speed distribution from the start) results in slower convergence and lower asymptotic performance, particularly for fast trajectories.

Reference

Yidong Zhu, Zibo Dai, Tongning Zhang, Leixin Chang, Hua Chen. **Spatiotemporal Agility: Time-Constrained Reinforcement Learning for Vision-Guided Dynamic Quadrupedal Interception.** arXiv:2608.06907, August 2026. <https://arxiv.org/abs/2608.06907>

Keywords: LLM safety • multilingual evaluation • Indic languages • cultural bias

SurakshaEval: An Indic Safety Benchmark for Multilingual LLMs

Human-Written Safety Prompts in Ten Indian Languages Reveal Systematic Over-Refusal and Missed Implicit Bias in Frontier Multilingual Models

By Debopriyo Banerjee, Kapil Rajesh Kavitha, Angana Borah, Xudong Han, Yuxia Wang, Parameswari Krishnamurthy, Utkarsh Agarwal, Atharva Kulkarni, Swaran Lata, Ayush Munot, Dhruv Sahnun, Aaryamonvikram Singh, Preslav Nakov, Monojit Choudhury

arXiv:2608.07862 • Preprint, August 2026

Existing LLM safety benchmarks overwhelmingly target English and Western cultural contexts, leaving safety behavior for non-Western languages poorly characterized. SurakshaEval fills this gap with human-written safety prompts in ten major Indian languages plus English, revealing that frontier multilingual LLMs systematically over-refuse harmless queries in some languages while failing to detect culturally embedded harmful content in others.

AT A GLANCE

Problem: LLM safety evaluation is English-centric; safety behavior in Indian languages with over one billion speakers is unmeasured.

Why it matters: Deployment gaps in non-English safety lead to either over-censorship or undetected harm at massive scale.

Method: Human-written prompts in 10 Indian languages + English, combining generic and culturally-specific items; evaluated on frontier multilingual LLMs.

Result: Three systematic failure modes identified: over-refusal, missed implicit bias, and insufficient cultural contextual awareness.

Evidence: Evaluated across leading multilingual LLMs; failures are consistent across model families.

The Big Picture

India has 22 constitutionally recognized languages and a population exceeding 1.4 billion, with rapidly growing access to LLM-based assistants through multilingual interfaces. Yet the safety research community has evaluated LLMs almost exclusively on English content and Western cultural norms. This creates a structural gap: models

deployed in India may be over-refusing benign requests in Hindi, Tamil, or Bengali because safety filters trained on English toxicity patterns generalize poorly to structurally different languages—while simultaneously failing to flag genuinely harmful content encoded in culturally specific references that the models lack context to recognize.

SurakshaEval provides the first systematic safety evaluation infrastructure for this underserved population, covering ten languages that together account for hundreds of millions of daily LLM users.

Why Existing Approaches Fall Short

Standard safety benchmarks (e.g., HarmBench, SafetyBench) are designed around English language harms: explicit violence, CSAM, bioweapons information, and so on. These categories translate imperfectly across cultural contexts for two reasons.

First, many harmful categories are culturally encoded. Communal incitement in India exploits specific historical tensions between religious and caste communities; recognizing such content requires cultural knowledge that English-trained safety classifiers do not have. Second, safety filters trained on English data often **over-generalize** to other languages, flagging content as unsafe because of surface-level patterns (vocabulary, script, topic) rather than actual harm.

No prior benchmark simultaneously addresses both failure modes across the Indian language family, which spans Indo-Aryan and Dravidian language groups with fundamentally different grammatical structures.

Core Mechanism

SurakshaEval is constructed in two tiers:

Generic prompts: Safety scenarios applicable across India and comparable to standard English safety benchmarks:

- Explicit violence and self-harm
- Misinformation and health misinformation
- Illegal activities
- Hate speech targeting national/ethnic identity

Culturally specific prompts: Scenarios that require knowledge of Indian sociocultural context to evaluate correctly:

- Communal tensions and religious incitement
- Caste-based discrimination and slurs
- Regional political incitement
- Language-community tensions

All prompts are written by **native speakers** of the target language with subject-matter expertise in the cultural safety context.

Technical Formulation

Let L be a language from the set of 11 languages (10 Indic + English), P a prompt, M a multilingual LLM, and $R = M(P)$ the model’s response. Safety evaluation assigns a label $y \in \{0, 1\}$ (safe/unsafe) to R .

Two metrics are computed per language L :

Over-refusal rate:

$$\text{ORR}(M, L) = \frac{|\{P \in \mathcal{P}_{\text{safe}}^L : y(M(P)) = 1\}|}{|\mathcal{P}_{\text{safe}}^L|}$$

i.e., the fraction of safe prompts incorrectly refused.

Harm detection rate:

$$\text{HDR}(M, L) = \frac{|\{P \in \mathcal{P}_{\text{unsafe}}^L : y(M(P)) = 1\}|}{|\mathcal{P}_{\text{unsafe}}^L|}$$

i.e., the fraction of harmful prompts correctly flagged.

An ideal model maximizes HDR and minimizes ORR across all L .

Learning or Inference Procedure

SurakshaEval is an **evaluation benchmark**; no training is performed. Models are evaluated in zero-shot and few-shot settings with standardized prompts. Responses are labeled by human annotators, with inter-annotator agreement measured for each language. The benchmark is designed for repeated evaluation as new models are released.

What the Guarantee Says

The benchmark provides two guarantees by construction: (1) all prompts are human-written by native speakers, ensuring linguistic authenticity; (2) the two-tier structure (generic + culturally specific) ensures coverage of both universal and India-specific harm categories. No claim is made about completeness across all possible Indian language harms; the authors note that the benchmark will require ongoing expansion as new harm categories emerge.

Experimental Findings

Three systematic failure modes are observed across all tested LLMs:

1. **Over-refusal:** Models refuse harmless queries in Indian languages at higher rates than equivalent English queries. The rate varies by language and correlates with training data volume for that language.
2. **Missed implicit bias:** Models fail to detect harmful content in culturally specific prompts—communal incitement, caste-based slurs, regional political hate speech—at significantly higher rates than for equivalent English harms.

3. **Insufficient contextual awareness:** Even when models detect a sensitive topic, responses often apply Western-centric framing that is inappropriate for the Indian cultural context, leading to responses that are technically safe but practically harmful or unhelpful.

Ablations and Interpretation

Generic vs. culturally specific items. Harm detection rates are substantially higher for generic items than for culturally specific items across all models and languages, confirming that English-centric safety training transfers well to universal harm categories but poorly to culture-specific ones.

Script vs. language effects. Languages written in non-Latin scripts (Devanagari, Tamil, Bengali, etc.) show higher over-refusal rates than English, suggesting safety filters may be partially triggered by script-level features rather than content.

Model size. Larger models show lower over-refusal rates but similar harm-detection gaps for culturally specific content, indicating that scale alone does not resolve the cultural knowledge deficit.

Reference

Debopriyo Banerjee et al. **SurakshaEval: An Indic Safety Benchmark for Multilingual LLMs.** arXiv:2608.07862, August 2026. <https://arxiv.org/abs/2608.07862>

Keywords: domain adaptation • time series • path signatures • distribution alignment

CPDA: Class-Conditional Path Distribution Alignment for Unsupervised Time-Series Domain Adaptation

A Composite Signature-Spectral Kernel Beats 30 Baselines on 13 Time-Series DA Benchmarks Without Adversarial Training

By Felix Ott, Christopher Mutschler

arXiv:2608.09193 • Preprint, August 2026

Most unsupervised time-series domain adaptation (UTS-DA) methods align only global marginal feature distributions, discarding the class structure that defines decision boundaries. CPDA instead aligns class-conditional latent *path distributions* via a novel composite signature-spectral kernel that jointly captures temporal trajectory structure, frequency content, and semantic features. Evaluated on 13 benchmarks against 30 baselines, CPDA achieves consistent state-of-the-art results with CNN, ResNet18, and TCN backbones—without adversarial training.

AT A GLANCE

Problem: UTS-DA distribution shifts arise from different users, sensors, or temporal dynamics; prior methods align only global marginals, losing class structure.

Why it matters: Labeling data for every new deployment context is infeasible; UTS-DA enables transfer without target labels in IoT, health, and manufacturing.

Method: Class-conditional alignment via a composite kernel combining path signatures, spectral features, semantic pooling, and low-rank dynamics.

Result: State-of-the-art on 13 benchmarks against 30 baselines with 3 backbone architectures.

Evidence: Non-adversarial; stable training across all experimental configurations.

The Big Picture

Time-series classifiers deployed in the real world face a pervasive problem: the distribution of training data differs from the distribution at deployment. A human activity recognition model trained on data from a 25-year-old male with one wearable device may perform poorly on a 60-year-old female with a different device. A machine fault-detection system trained on one production line may fail on a nominally identical line with slightly different

resonance characteristics.

Unsupervised time-series domain adaptation addresses this by transferring knowledge from a labeled **source domain** to an unlabeled **target domain**. The key insight of CPDA is that temporal data has rich structure—**path trajectories** in feature space, **frequency spectra**, and **class-specific distributional signatures**—that global marginal alignment discards. CPDA preserves this structure through a principled kernel design.

Why Existing Approaches Fall Short

The dominant paradigm in UTS-DA follows the deep domain adaptation tradition from computer vision: minimize a distributional discrepancy (e.g., MMD) or train a domain adversary (e.g., DANN) on marginal feature distributions. Both approaches have structural limitations for time-series data.

Global marginal alignment ignores class structure. If source class A aligns with target class B due to superficial distributional similarity, a classifier that looked correct in source space will fail in target space. This **negative transfer** effect is well-known but remains underaddressed in UTS-DA.

Spectral and temporal structure is discarded when features are pooled before alignment. A model that misses the difference between a 10 Hz and a 15 Hz vibration pattern may align the two classes' pooled statistics while failing on the actual classification problem.

Adversarial training is unstable, sensitive to hyperparameters, and prone to mode collapse, making it difficult to deploy reliably across diverse benchmark settings.

Core Mechanism

CPDA operates in the encoder's latent space, treating each sample as a **path**—a sequence of latent feature vectors over time. It aligns class-conditional path distributions via a composite kernel k_{CPDA} that combines four components:

1. **Semantic pooling kernel:** Classic RBF kernel on time-averaged latent features; captures overall distributional similarity.
2. **Path-signature kernel:** Measures similarity of ordered polynomial statistics of the time-series trajectory—captures temporal dynamics.
3. **Spectral kernel:** Fourier-based kernel on the frequency-domain representation of the latent path—captures periodic structure.
4. **Low-rank path-signature kernel:** Compressed, dominant modes of temporal variation via truncated signature expansion—reduces noise sensitivity.

Technical Formulation

Let $\phi_\theta : \mathcal{X} \rightarrow \mathbb{R}^{T \times d}$ be the encoder mapping a time-series x to a T -step latent path. Define the class-conditional source and target distributions:

$$P_S^y = \{\phi_\theta(x) \mid x \in \mathcal{D}_S, y_x = y\}, \quad P_T^y = \{\phi_\theta(x) \mid x \in \mathcal{D}_T, \hat{y}_x = y\}$$

where $\hat{y}_x$ are pseudo-labels for target samples.

The CPDA discrepancy is:

$$\mathcal{D}_{\text{CPDA}} = \sum_{y=1}^C \text{MMD}_{k_{\text{CPDA}}}^2(P_S^y, P_T^y)$$

with:

$$k_{\text{CPDA}}(\gamma, \gamma') = \alpha_1 k_{\text{sem}}(\gamma, \gamma') + \alpha_2 k_{\text{sig}}(\gamma, \gamma') + \alpha_3 k_{\text{spec}}(\gamma, \gamma') + \alpha_4 k_{\text{lr}}(\gamma, \gamma')$$

where γ, γ' are latent paths and α_i are learned mixture weights.

The training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\mathcal{D}_S) + \lambda \mathcal{D}_{\text{CPDA}}(\mathcal{D}_S, \mathcal{D}_T)$$

Learning or Inference Procedure

Pseudo-label generation: Target pseudo-labels are produced by the source-trained classifier and updated iteratively as the encoder improves. High-confidence pseudo-labels (above a threshold) are used for alignment.

Optimization: Stochastic gradient descent on $\mathcal{L}$; no adversarial loop, no discriminator. The MMD-based objective is smooth and differentiable.

Inference: Standard forward pass through the adapted encoder and classifier; no test-time computation overhead relative to the base model.

What the Guarantee Says

Under standard domain adaptation assumptions (covariate shift, shared label conditional), minimizing class-conditional MMD drives the target encoder toward the source class-conditional distributions, providing a theoretical basis for classifier transfer. The path-signature kernel is characteristic on path space under mild regularity conditions, guaranteeing that $\mathcal{D}_{\text{CPDA}} = 0$ implies equality of path distributions. In practice, finite-sample approximation introduces error, but the rich kernel reduces the approximation gap relative to simpler kernels.

Experimental Findings

- **13 benchmarks:** Including activity recognition (HAR, UCIHAR, WISDM), fault detection (CWRU, PU), and physiological monitoring (EEG, ECG).

- **30 baselines:** Spanning discrepancy-based (MMD, CORAL), adversarial (DANN, CDAN, AdvSKM), and pseudo-labeling methods (SHOT, SDAT).

- **3 backbones:** CNN, ResNet18, TCN—CPDA achieves state-of-the-art with all three.

Margin: CPDA outperforms the second-best method on average across all 13 benchmarks by a consistent margin with all backbone choices.

Ablations and Interpretation

Kernel component ablation. Removing any single component (k_{sig} , k_{spec} , k_{lr} , k_{sem}) degrades performance on most benchmarks, confirming that each component captures complementary distributional information. The path-signature kernel has the largest marginal contribution on benchmarks with strong temporal dynamics (fault detection); the spectral kernel dominates on benchmarks with periodic signals (EEG, ECG).

Global vs. class-conditional alignment. Replacing class-conditional alignment with global marginal MMD using k_{CPDA} degrades performance substantially on benchmarks with closely adjacent class clusters, confirming the negative-transfer hypothesis.

Non-adversarial vs. adversarial. CPDA trains stably across all 13 benchmarks; GAN-based methods (DANN, CDAN) show higher variance and failed convergence on 3 of 13 benchmarks.

Reference

Felix Ott, Christopher Mutschler. **CPDA: Class-Conditional Path Distribution Alignment for Unsupervised Time-Series Domain Adaptation.** arXiv:2608.09193, August 2026. <https://arxiv.org/abs/2608.09193>